BIS Innovation Hub

London Centre

Project Kairos: Statement of Work

1. Introduction

Project Kairos will explore how reserves, deposits, and securities can be tokenised and moved across DLT ledgers. It will examine the nuances of issuance, settlement, and secure interoperability of tokens exploring the feasibility of different use cases to test protocol capabilities and bridge design trade-offs.

2. Objectives

The main objectives of Project Kairos are:

Objective 1: Assess if central banks could safely and efficiently tokenise reserves on private permissioned DLT ledgers to enable efficient settlement and enhanced cross-ledger interoperability. This includes testing ways to control tokenised reserves on permissioned DLT ledgers (e.g. ability to freeze and/or repatriate as needed), while identifying minimum technical requirements to do so. This objective would cover similar questions for tokenised assets mobilised as collateral, remaining under the control of central banks.

Objective 2: Test how DLT bridges could transfer tokenised deposits and securities across different private permissioned and public permissionless DLT ledgers, while retaining programmable token logic across different ledgers (e.g. access controls for tokenised reserves and coupon payments for tokenised securities).

Objective 3: Explore different ways to design bridges depending on central bank preferences and policy frameworks and compare them (e.g. any different design choices for DLT bridges, DLT ledgers and tokenised securities).

Objective 4: Consider how this model compares to other potential models for central bank interoperability. For instance, consider the relative benefits of synchronisation (as tested in Meridian project series) and integrated settlement (where both assets in a transaction are located on the same ledger). This will include a side-by-side comparison of settlement of equivalent complex use cases using 1) synchronisation with RTGS and 2) tokenised reserves and bridging.

3. Technical components

3.1. Token contracts

A token definition shall be created to fulfil the purposes of deposits, a digital bond, and reserves. This definition shall be agnostic to the underlying ledger and set out which data elements and functions combine to compose the token.

Mint functionality

Tokens may only be minted by the owner of the token contract, or upon delegation to the bridge, by the bridging entity. It is presumed that multiple versions of the mint function will be required to fulfil different designs of the bridging entity [see Bridging Entity]. These may include, but are not limited to, the inclusion of multi-party signatures or signed messages delivered by the bridge.

Burn functionality

Tokens may only be burned by the owner of the token contract, or via delegation, by the bridge. The same considerations for multiple variants discussed in mint functionality apply. It is presumed that upon a burn request, tokens will be placed in escrow, and a corresponding mint request will be sent across the bridge. Upon receipt of confirmation of successful mint, the tokens will be burned.

Transfer functionality

Tokens may be directly transferred by holders of those tokens. Additionally, tokens may be transferred on the behalf of the holder by another party, assuming they have been approved to transact [see Token allowance].

Approve list

Tokens may only be held by approved wallets. Strict allow listing shall be developed across all ledger types, enabling the token owner or a delegated authority to publish which wallets may hold tokens.

Token control

It is expected that the token issuer will retain the ability to control tokens that are issued. The issuer should be enabled to freeze the transfer of tokens. Additionally, the issuer should have a mechanism to repatriate tokens, crediting the token holders in the underlying system.

Token allowance

A robust delegated approval mechanism shall be included in the token design. It is assumed that this could mirror the EIP-2612 standard, allowing the bridging entity, and other approved delegates to transact on the token holder's behalf. While the "permitted entity" may not necessarily be on the approve list, any transfer taken by this entity must follow the transfer requirements.

Reconciliation

The project shall create a robust reconciliation mechanism between the total supply of tokens (across all ledgers) and their underlying balance. In the event of a reconciliation break, it is expected that logging within the underlying system (RTGS or commercial banks) and ledgers are sufficient to trace the origin of the break, and that mechanisms are in place to rectify the error.

Token service

It is envisioned that mechanisms to pay interest on reserves could be required. The token contract shall track the average daily balance of a wallet, crediting new tokens to the wallet based on the overnight interest rate (to be published by the central bank).

Digital assets also require services for both principal and interest (P&I) payments and for maturity. These payments may require a cross-chain transfer of either deposits or reserves from the issuer's account to holders of the digital asset.

3.2. Public permissionless ledgers

To meet the goals of the project, the token design shall be tested across multiple public permissionless ledgers. The token shall be tested across both an EVM and non-EVM based ledger. It is assumed that the test networks for Ethereum and Solana are in scope for the project, but the final ledgers used will be decided based on conversations with the selected vendor.

3.3. Private and permissioned ledgers

To further validate the flexibility of the token, the project shall test their deployment and interoperability within private permissioned ledgers. It is presumed that the bridging entity is not a participant in the validation of the ledger, but that the ledger is trusted to operate securely. At least two private or permissioned ledgers, such as Hyperledger Besu and Canton, shall be tested. It is assumed that the Besu ledger will need to be deployed and hosted within the BISIH cloud environment for the purposes of this project. As with the public ledgers, the selection includes both EVM-compatible and non-EVM chains to test portability across different architectures.

3.4. Bridging Entity

Bridges shall be developed to manage the transfer of tokens between various ledgers, and between the RTGS and ledgers. To enable the greatest learnings from the project, multiple variations of the bridge shall be developed.

Single operator bridge

This variant is intended to showcase a simplistic bridging model. In this model, a single entity would fully operate the via a proof of authority bridge and be assumed to be fully trusted.

Distributed bridge

Within the distributed model, it is assumed that a consortium of regulated entities i.e. commercial banks and the central bank would operate the validators of the bridge. Their consensus would be required for the bridge to act.

Single operator distributed bridge (Extension requirement, please scope as an "add-on")

An additional bridge variant is *optionally* in scope; this variant is a single operator distributed bridge, where a single legal entity operates a consensus driven bridge.

3.5. Other contracts and functionality

Reserve tokens, while a large part of the project, are not sufficient upon their own for the envisioned workflows to be tested. The below on-chain functionalities are required for the valid testing of the project.

Asset contract

A generic real world asset token in the form of a digital bond shall be created for the purpose of testing delivery versus payment (DvP) transactions or repurchase agreements. This token shall be bridgeable across ledgers in the same way as the reserve token and would be modelled to be issued by a third party.

Liquidity preferences (Extension requirement, please scope as an "add-on")

Entities that are approved to hold reserve tokens must be enabled to configure their preferences for the levels of liquidity across chains. The project is agnostic to whether this is stored on or off chain. These preferences enable the automated liquidity rebalancing and "smart sourcing" of liquidity.

4. Experiments

Using the developed assets described above. The project will test a variety of transactions, comparing how they operate across various ledger types and bridge models. The below test cases shall be developed and demonstrated by the vendor across the range of ledgers and bridge models.

4.1. Liquidity transfer

A liquidity transfer refers to the transfer of tokens from one ledger to another. In this test, the bridge must demonstrate the ability to burn tokens, transfer confirmation of the burn, and mint tokens on a secondary ledger. Note, this transfer refers to both the transfer of CBm tokens and asset tokens.

4.2. Delivery versus payment (DvP)

Delivery vs payment (DvP) transactions refer to the exchange of reserve tokens with an asset token. This experiment will test the ability to combine a liquidity transfer with a DvP transaction, sourcing the asset tokens from one chain, before conducting an atomic swap on the chain that holds the reserve tokens.

4.3. Repurchase agreements (Repo)

A repurchase (repo) facility shall be created for the central bank's operations. This facility enables commercial banks to source reserves in exchange for the locking of eligible collateral. This test

shall demonstrate the cross-chain bridging of eligible securities into the ledger operating the repo facility, before atomically conducting a repo transaction.

4.4. RTGS bridge to ledger

To mint a new token on an external ledger, a corresponding transaction from a bank to a technical account within the RTGS is first required. The RTGS will generate a confirmation of deposit, signed by the RTGS. The bridge may then use this confirmation of deposit to mint tokens on a ledger.

4.5. Ledger bridge to RTGS

To "redeem" tokens for CBm within the RTGS, the contract owner shall be enabled to escrow tokens. Upon the escrow of the tokens, the bridge shall instruct the RTGS to transfer CBm from the technical account to the bank's account. Once confirmation of successful transfer has been received, the contract owner will burn the tokens from the ledger.

4.6. Automated liquidity rebalancing (Optional, please quote separately)

Using the [Liquidity preferences], the bridge should be enabled to automatically rebalance account positions across ledgers and the RTGS. This is intended to be triggered by balances above or below user preferences, or at specified times of the day.

4.7. "Smart sourcing" of liquidity (Optional, please quote separately)

Using the [Liquidity preferences], the bridge should provide programmability for the source of liquidity that is used to settle a transaction.

5. Non-functional requirements

5.1. Privacy

While not a specific focus of this project. It is desirable that the bridge and contracts developed within this work be examined for potential future privacy requirements. This entails that contracts and bridges should be extendible to enable privacy in a potential future phase.

5.2. Security

This project is a proof of concept, intended to provide learnings for the central banking community. As such, it will be conducted within a private, secured cloud environment. It will interact solely with test versions of public networks, with no "real money" transactions. It is

expected that, while this proof of concept is within a secure environment, the project will showcase best practices for key management, smart contract design, and bridge security.

5.3. Speed and throughput

There are no defined speed and throughput requirements for this project. It is expected that efforts are made to enable high levels of speed and throughput within the bridge, and that these metrics may then be assessed across various bridge designs and network selections.

6. Governance

6.1. Collaborative design

This statement of work (SoW) has been designed to outline, in more detail, the requirements for the selected vendor for project Kairos. The BIS Innovation Hub strongly values the experience of the vendor community and welcomes challenges and suggestions to the requirements outlined in this SoW. Any amendments to this SoW will be discussed and agreed upon prior to the issuance of a contract.

6.2. Code compliance and licensing

All open-source library usage, code quality standards, and third-party dependency requirements shall be governed by Annex 1 and Annex 2.

6.3. Existing assets

Through prior work at the BIS Innovation Hub, an emulated version of the UK RTGS system has been created. This emulator has capabilities for the reservation and transfer of funds via an API interface. This RTGS emulator will be available to the selected vendor for use during the project.

6.4. Development location

The selected vendor will be expected to use the BISIH's Azure cloud for all development activities. This includes utilization of Azure DevOps for source control, CI/CD pipelines, and work item tracking. Access will be provided via either B2B account access, or by the issuance of development machines to the development team.

7. Deliverables

1) Technical deliverables
   a) Contracts
      i) Security contract – to emulate a digital bond
      ii) Reserve contract – to emulate tokenised reserves
      iii) Deposit contract – to emulate tokenised deposits
      iv) Repurchase contract – to emulate a repurchase contract between the central bank and a commercial bank
      v) Settlement contract(s) – to enable HLC atomic swaps on chain and DvP
      vi) Registry, Identity and Compliance contract(s)– to deploy tokens, manage participants and allow/deny lists, verify identities, enforce transfer rules
      vii) Governance and Admin contract(s) – to manage roles, parameters and emergency actions
      viii) **Note on Contract design:** The contracts outlined above represent the current envisioned architecture. However, alternative design patterns and approaches remain welcome, subject to consultation and mutual agreement. In particular, (a.vi) and (a.vii) may be modified, combined or omitted entirely depending on the final agreed architecture and the specific requirements for each ledger implementation. This recognises that design patterns may vary significantly across ledger platforms.
   b) Contracts Validation and Audit
      i) Each (a) contract shall include:
         (1) Unit Tests (>95% coverage)
         (2) Integration tests with all cross-contract workflows
         (3) Fuzz tests shall be performed on all critical functions (transfer, calculations, access control) with no unexpected failures
         (4) All tests shall execute against local deployment only, no external dependencies permitted
         (5) Deployment scripts shall support local testing and final delivery environments
      ii) All contracts are expected to undergo independent third-party security review and audit. Such review and audit will be commissioned and procured separately and are outside of the scope of this engagement. Contracts shall be delivered in an audit-ready state at agreed milestones, with findings remediated prior to the final acceptance.
   c) Ledgers
      i) Ledgers networks:
         (1) Ethereum: Sepolia
         (2) Solana: Testnet
         (3) Hyperledger Besu (Modelled to be operated by a group of regulated entities): Private Testnet
         (4) Canton: Private Test Domain or Canton Network
      ii) Contracts (a.vi) and (a.vii) subject to design of tokens
      iii) Contracts (a.i), (a.v), (a.vi), (a.vii) shall be deployed on all ledgers
      iv) Additionally, contracts shall be deployed on the following ledgers:
         (1) Ethereum: (a.iii), (a.iv)
         (2) Solana: (a.iii), (a.iv), (a.vi)
         (3) Hyperledger Besu: (a.ii), (a.iv)
         (4) Canton: (a.ii), (a.iv)

d) Bridge and Governance Configurations
   i) Centralised (PoA)
   ii) Decentralised
      (1) Modelled to be operated by a group of regulated entities
      (2) **Extension:** Operated by a single legal entity
e) Orchestrator
   i) An off-chain entity responsible for orchestrating bridge operations
   ii) Transaction status tracking and monitoring
f) User interface
   i) A user interface for the orchestrator to view the status and lifecycle of operations (Orchestrator Dashboard)
   ii) A user interface for users (participants, regulators, authorities) to initiate, track, monitor and view the status of transactions (Participant and Authority Portals)
g) Visualisation tool
   i) A UI portal to view (in a step-by-step manner) the stages of a transaction. This shall enable demonstration to senior leadership by providing understanding of the balances of individual wallets, and the transfer(s) of tokens between them. It shall include a comprehensive audit trail capturing all token lifecycle events (minting, transfers, burns, reservations etc.) with timestamps, actor identification, transaction references and exportable reports.

2) Documentation
   a) The selected vendor shall deliver the below technical documentation. For the avoidance of doubt, these requirements are in addition to those noted in Annex 1.
      i) Technical documentation on the solutions architecture
      ii) Technical documentation on the smart contracts developed
      iii) Technical documentation on the relative differences in the smart contracts deployed between ledgers of different technical architectures
      iv) Technical documentation on the bridge designs
      v) Technical documentation on the component design – including data models and behavioural diagrams (sequence, flow, state)
      vi) UI and Visualization tools may have minimal technical documentation, to cover deployment and maintenance specifications.
   b) The selected vendor shall deliver the below non-technical documentation
      i) Comprehensive documentation on business requirements developed within the project
      ii) Comprehensive documentation on the use of open-source software incorporated within the project (as dependencies and or linked libraries)
      iii) Detailed test plans to cover the use cases (experiments) included in this SoW
      iv) Detailed test plan results
3) Report development
   a) The selected vendor is expected to co-write the technical annex to be included in the project report. This is to include workflow diagrams and descriptions, in addition to comparative analyses of bridge designs and token designs. It shall also include descriptions of the design decisions made within the project, and a justification as to why the project made technical choices.
4) Development standards
   a) The selected vendor shall comply with BISIH code standards for a T2 project included in Annex 1 of this SoW

b) All source code shall be version-controlled using DevOps Git with meaningful commit messages and branching strategies
c) Code shall be modular, maintainable and follow separation of concerns principles
d) Code reviews shall be conducted by at least one qualified reviewer other than the author. Reviews shall verify compliance with coding standards, adherence to industry guidelines and best practices, functional correctness, adequate test coverage, and sufficient documentation
e) The vendor shall adhere to recognised industry standards, frameworks, and the best practices applicable to blockchain development, including platform-specific standards and guidelines
f) Unit tests shall be automated and cover normal operation, edge cases, and error conditions. All tests shall be included in DevOps CI/CD pipelines and executed automatically on each build. Test results and coverage reports shall be provided with each delivery

Annex 1: Code Compliance and Code Quality

The following minimal quality guidelines must be met as part of the implementation of the solution.

1. comprehensive "readme" documentation, including Project Code description and goals, architecture diagrams, installation instructions, test execution guidance, development tools, and deployment system dependencies.

2. preparing the Project Code repository for distribution by removing environment-specific information.

3. implementing testing as follows:

    1. Poc/Prototype (Tier 2): Minimum 20% Project Code coverage of non-UI functional requirements.

    2. At least one end-to-end test covering non-UI functional requirements must be implemented.

4. managing dependencies as follows:

    1. conduct an inclusion assessment for all dependencies based on assessment template provided by BISIH.

    2. consulting BISIH before adding dependencies that do not meet the criteria set out in Annex 2.

    3. cooperating with the BIS in performing licence and vulnerability scans and taking remediation actions:

    i. remediating licence compliance issues and preparing compliance artifacts.

    ii. Tier 2: High and critical dependency security issues must be remediated.

Annex 2: Dependency Licensing

The table below shows which types of dependencies, based on their licences, can be accepted for each type of Project-Code execution environment.

Snippet, or code with no known Execution Environment	**Library**	**Server**	**Application and Universal Deployment**
Dependencies under permissive licences	Dependencies under permissive or weak copyleft licences	Dependencies under permissive or weak copyleft licences	Dependencies under permissive or weak copyleft licences

The use of dependencies under strong copyleft licences, SaaS licences, or proprietary licences is generally discouraged. Strong copyleft licences, such as the GNU General Public License (GPL), impose strict requirements that may require the entire Project Code to be distributed under the same licence, limiting flexibility. Proprietary licences often restrict modification or redistribution, which can conflict with the BIS's goal of sharing Project Code as a public good.